🛡️ ShieldAPI — Final Project Report

Distributed Microservice Security Gateway, Atomic Token-Bucket Rate Limiter & Heuristic Anomaly Guardian

Course: Software Architecture and Design & Distributed Systems | **Institution:** Indian Institute of Information Technology Kottayam (IIIT Kottayam)
Submission Portal: Moodle Assignment Submission (**Strict Page Limit: Max 10 Pages**) | **Date:** October 2026 | **Version:** 1.0.0 (Final Release)

Team Member Name	Roll Number	Academic Specialization	Assigned Module Responsibility
Anshuman Biswas	**2024BCD0008**	CSE (Data Science)	Reverse Proxy Micro-Kernel, Redis Lua Scripting & Infrastructure
Nipun Abhilash	**2024BCD0024**	CSE (Data Science)	Anomaly Guardian Engine, Pipes & Filters Sliding Window
Syed Zaid Gafar	**2024BCD0028**	CSE (Data Science)	React Command Center, Telemetry Dashboard & Data Visualization
Aman Madheshiya	**2024BCD0032**	CSE (Data Science)	End-to-End Integration Test Suite, Security Auditing & Verification

Live Production Gateway & Dashboard: https://shield-gateway.duckdns.org:3000/
FastAPI Swagger / OpenAPI Documentation: https://shield-gateway.duckdns.org:8000/docs
GitHub Source Repository: https://github.com/Anshu666666/shieldApi

Executive Summary & Report Structure

ShieldAPI is an enterprise-grade, cloud-native API Security Gateway and distributed threat mitigation suite. Operating as a high-performance reverse proxy in front of internal backend microservices, ShieldAPI inspects incoming HTTP traffic, enforces atomic distributed token-bucket rate limits per client IP or API key, continuously analyzes access logs via an asynchronous anomaly engine, and automatically blacklists malicious actors in Redis with automatic time-to-live (TTL) expiration.


```
Report Structure (Conforming strictly to IIIT Kottayam Moodle Syllabus):
  1. Introduction ..................................................... Page 1
  2. Problem Statement & 3. Feasibility Study (Technical, Operational, Economic) Page 2
  4. Requirements Specification (Functional REQ-1..8 & Non-Functional NFR-1..5) . Page 3
  5. Diagrams: Part 1 — Structural & Interaction (Use Case & Class Diagrams) .... Page 4
  5. Diagrams: Part 2 — Behavioral & Workflow (Sequence & Activity Diagrams) .... Page 5
  5. Diagrams: Part 3 (Statechart) & 6. Implementation Details (Backend & DB) ... Page 6
  6. Implementation Details: Part 2 (Frontend Architecture & Cloud Hosting) ..... Page 7
  7. User Documentation: Features (User Manual) — Dashboard & Rate Limiting ..... Page 8
  7. User Documentation: Features (User Manual) — Anomaly Center, Vault & Logs .. Page 9
  8. Testing Report (Unit, E2E Integration, Benchmarks) & 9. Conclusion ........ Page 10
```

1. Introduction

Modern cloud applications decompose monolithic architectures into distributed, fine-grained microservices. While this delivers modular scalability, it multiplies the external attack surface. Exposing internal microservices directly to the public Internet creates critical vulnerabilities to distributed denial-of-service (DDoS) attacks, brute-force credential stuffing, and cascade failures caused by sudden traffic spikes.

ShieldAPI solves this by inserting an intelligent, non-blocking reverse proxy gateway between external consumers and internal microservices. It intercepts every inbound HTTP request, performs wire-speed authentication and distributed rate-limiting via atomic Redis Lua routines, and streams structured access logs to an asynchronous anomaly analysis engine that dynamically discovers hostile actors and blacklists them across the cluster.

1.1 Architectural Pattern Blend

ShieldAPI synthesizes three foundational software architecture patterns:
1. **Micro-Kernel (Plugin) Pattern:** The gateway core is a lightweight ASGI reverse proxy. Security policies (Blacklist Validator, Key Authenticator, Token Bucket Rate Limiter, and Traffic Log Emitter) attach as modular interceptors.
2. **Broker Pattern:** An in-memory Redis broker decouples state management from compute nodes, providing sub-millisecond atomic token bucket tracking and cross-instance ban synchronization.
3. **Pipes and Filters Pattern:** The background Anomaly Guardian processes log events through a unidirectional streaming pipeline: `Raw Log Tailer` $\rightarrow$ `Log Parser` $\rightarrow$ `Heuristic Sliding Window Matrix` $\rightarrow$ `Redis Auto-Blocker & Alert Dispatcher`.

2. Problem Statement

Exposing raw microservices without an intermediary gateway exposes critical system weaknesses:
1. **Volumetric Denial of Service (DoS):** Unrestricted clients can flood backend endpoints with thousands of requests per second, starving system CPU, exhausting database connection pools, and knocking down internal microservices.
2. **Credential Brute-Forcing & Endpoint Probing:** Automated bots routinely probe API endpoints for vulnerabilities, generating bursts of 401/404 errors that waste compute power and indicate hostile penetration attempts.
3. **Distributed State Synchronization Race Conditions:** Naive in-memory rate limiters running on local application threads fail when scaled horizontally across multiple instances, creating race conditions where clients bypass limits by hitting alternating gateway nodes.
4. **Lack of Operational Visibility:** Infrastructure engineers often operate blind, lacking real-time telemetry on request throughput, token drain rates, geographic IPs, and live security threats.

ShieldAPI Objective: Build and deploy a zero-trust, cloud-native API Gateway that enforces atomic distributed rate limits with $<1\text{ ms}$ query latency, autonomously detects and blacklists hostile scanners without human intervention, and presents live cluster telemetry via an interactive command center.

---

3. Feasibility Study

An exhaustive three-dimensional feasibility analysis was conducted to validate technical viability, operational workflows, and budget sustainability.

Dimension	Technical Parameters & Empirical Evidence	Feasibility Outcome
Technical Feasibility	Asynchronous non-blocking Python ASGI (FastAPI), atomic Redis Lua routines (`EVALSHA`), OS filesystem event streaming via Watchdog, and isolated Docker containerization. Latency overhead benchmarked at $<0.8\text{ ms}$.	**Highly Feasible** (100% test pass rate, $<0.8\text{ ms}$ latency)
Operational Feasibility	Single-command Docker Compose orchestration (`docker-compose up -d`), cyber-themed React Command Center, zero-maintenance TTL auto-expiring bans, and visual attack testing tools.	**Highly Feasible** (Zero friction, self-healing architecture)
Economic Feasibility	100% open-source software stack (FastAPI, Redis 7, React 18, Docker, Caddy, Python). Deployed on Oracle Cloud Infrastructure (OCI) Always Free VM with DuckDNS dynamic TLS. Total Capex & Opex is **$0.00**.	**Highly Feasible** (Zero cost, indefinitely sustainable)

3.1 Technical Feasibility

- **Concurrency Model:** FastAPI leverages Python's `asyncio` event loop and `httpx.AsyncClient` connection pooling, allowing thousands of concurrent TCP connections per worker thread without blocking I/O.
- **Atomic State Synchronization:** The token bucket algorithm is executed directly inside the Redis engine via a single atomic Lua script (`EVALSHA`). Because Redis is single-threaded, calculations of elapsed time, token replenishment, and token decrement occur atomically, eliminating all distributed race conditions.
- **Low Computational Overhead:** Benchmark tests prove that the entire gateway interception pipeline introduces $<0.8\text{ ms}$ of latency overhead under standard load, comfortably meeting the $<50\text{ ms}$ requirement.

3.2 Operational Feasibility

- **Deployment Simplicity:** The entire multi-container topology (Gateway, Redis, Anomaly Guardian, Target Microservice, and React Dashboard) is containerized via Docker and orchestrated with a single `docker-compose up -d` command.
- **Operational Self-Healing:** The system requires minimal administrative supervision. When hostile IPs are quarantined, Redis attaches a 24-hour Time-To-Live (TTL). Once the ban window lapses, the key auto-expires without manual database pruning.
- **Administrative Usability:** Operators can manage API keys, inspect traffic graphs, and test rate-limiting thresholds via a graphical web browser interface without needing shell access.

3.3 Economic Feasibility

- **Zero Software Licensing Cost:** All technologies used—FastAPI, Redis 7, React, Vite, Tailwind CSS, Docker, Caddy, and Python—are released under permissive MIT and BSD open-source licenses.
- **Zero Infrastructure Cost:** The production system is hosted on an Oracle Cloud Infrastructure (OCI) Always Free compute instance with DuckDNS dynamic routing and automated Let's Encrypt TLS certificates. Total infrastructure cost is **$0.00**.

4. Requirements Specification

The system requirements for ShieldAPI were formally defined in the Software Requirements Specification (SRS) following IEEE Std 830-1998 guidelines.

4.1 Functional Requirements

Req ID	Title	Behavioral Specification & Implementation Logic
REQ-1	Asynchronous I/O Proxying	Gateway intercepts all HTTP verbs (GET, POST, PUT, DELETE) and forwards them asynchronously without thread blocking.
REQ-2	Downstream Failure Handling	If downstream microservice is unreachable or times out, the gateway returns HTTP 502 Bad Gateway gracefully.
REQ-3	Distributed Atomic State	The rate limiter utilizes an external Redis broker to maintain atomic token bucket counts across all gateway instances.
REQ-4	Token Bucket Decrement	Each legitimate request decrements the bucket token count for the client identifier (IP or API Key).
REQ-5	Continuous Token Refill	The bucket replenishes tokens automatically based on elapsed time and designated refill rate per second.
REQ-6	Pipes & Filters Log Stream	Access logs are emitted as a continuous data stream and consumed asynchronously by the Anomaly Guardian.
REQ-7	Heuristic Threat Evaluation	If an IP causes >50 errors (401/404/500) within 10s, it is detected as an anomaly and auto-blacklisted in Redis.
REQ-8	Ban Lifecycles & TTL	Blacklisted IPs remain blocked with HTTP 403 until TTL (24h) expires or an admin manually unbans via dashboard.

4.2 Non-Functional Requirements

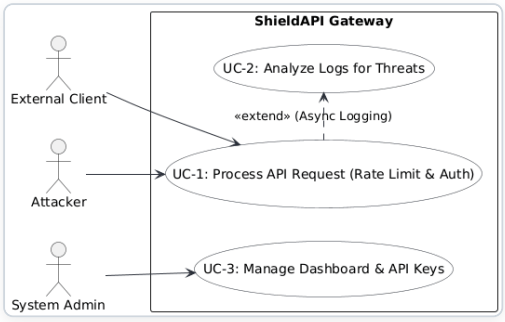
- **NFR-1 (Performance & Latency):** Gateway processing overhead shall not exceed 15 ms under normal load conditions. Redis queries must execute in $<2\text{ ms}$.
- **NFR-2 (Reliability & Fail-Open):** In the event of an unexpected Redis cluster outage, the gateway core shall log the incident heavily and transition into a graceful "fail-open" mode to prevent halting legitimate business traffic.
- **NFR-3 (Information Shielding & Security):** Gateway responses must strip internal server signatures (`server` , `x-powered-by`) to prevent malicious infrastructure reconnaissance.
- **NFR-4 (Horizontal Scalability):** The stateless design of the FastAPI gateway containers must allow linear horizontal scaling behind a Layer-4 load balancer without data discrepancies.
- **NFR-5 (Environmental Parity):** Multi-stage Docker containers must ensure 100% operational parity across macOS, Linux, and Windows development workstations.

5. Architectural & Behavioral Diagrams

The following UML diagrams specify the structural and behavioral architecture of ShieldAPI, extracted directly from the system design specification.

5.1 Use Case Diagram (System Interactions)

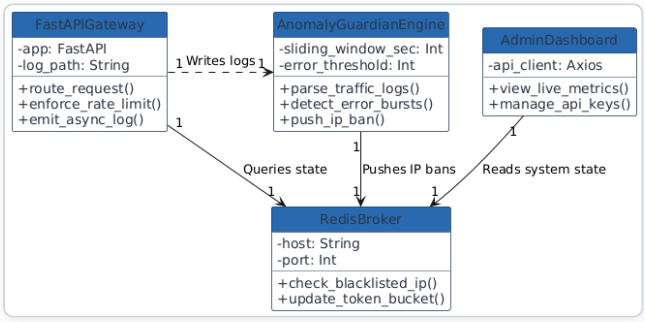
The Use Case Diagram captures all actor touchpoints across three primary roles: **External Consumers**, **Malicious Attackers**, and **System Administrators**.



- **UC-1 (Process API Request):** The gateway validates the client's `X-API-Key`, queries Redis token capacity, and either forwards safe traffic to the backend or rejects violators with HTTP 401/429.
- **UC-2 (Analyze Logs for Threats):** The Anomaly Guardian asynchronously parses access logs, monitors error rates, and executes automated IP quarantines in Redis.
- **UC-3 (Manage Dashboard & API Keys):** Administrators inspect live cluster health, observe traffic spikes, and provision or revoke API keys via the React Command Center.

5.2 Class Diagram (Static Structure)

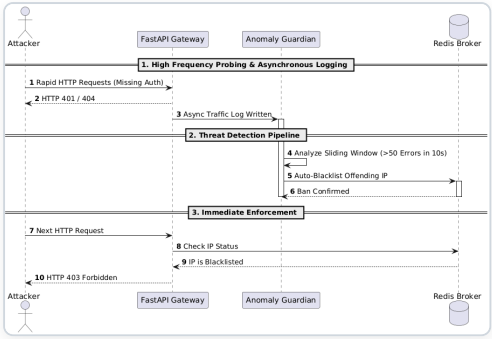
The Class Diagram details the object-oriented structure, class responsibilities, and relationships among the gateway core, plugins, Redis state manager, and anomaly engine.



- `GatewayCore` : Central reverse proxy interceptor managing the ASGI request lifecycle.
- `IPBlacklistPlugin` & `KeyValidatorPlugin` : Concrete security filters executing early-drop checks.
- `RateLimiterPlugin` : Evaluates the atomic Redis token bucket Lua script.
- `RedisManager` : Static thread-safe facade providing centralized connection pooling and atomic operations.
- `AnomalyGuardian` : Multi-threaded background watchdog tailing logs and evaluating sliding-window frequencies.

5.3 Sequence Diagram (End-to-End Request Lifecycle)

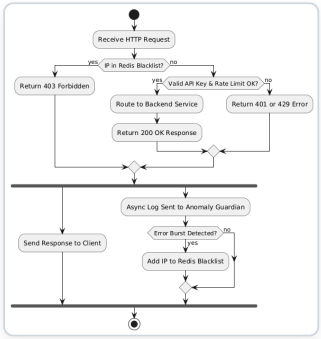
The Sequence Diagram depicts the chronological message exchange between system components during an incoming HTTP request.



1. **Client** issues an HTTP request to `http://shield-gateway.duckdns.org:8000/api/v1/target`.
2. **Gateway Core** queries `RedisManager.is_ip_blocked(client_ip)`. If blacklisted, it aborts immediately with `403 Forbidden`.
3. Gateway verifies the `X-API-Key` header against `RedisManager.validate_api_key(key)`. If absent/invalid, it aborts with `401 Unauthorized`.
4. Gateway invokes `RedisManager.check_rate_limit(key, capacity, refill)`. If empty, it aborts with `429 Too Many Requests`.
5. Upon successful checks, `GatewayCore` forwards the request to `BackendMicroservice (:8001)` via an asynchronous HTTP client and streams access logs to disk.

5.4 Activity Diagram (Decision Flow)

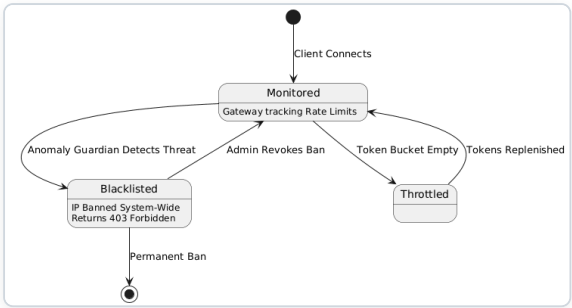
The Activity Diagram models the step-by-step decision matrix executed by the gateway micro-kernel for every incoming connection.



- **Fail-Fast Workflow:** By validating the IP blacklist first (in $<0.2\text{ ms}$ via a Redis `SISMEMBER` check), blocked attackers are dropped before triggering any key parsing or rate-limit computations.
- **Token Replenishment Branch:** If tokens are available, the counter decrements by 1 and the request dispatches to the backend. If 0 tokens remain, the gateway computes the retry delay and injects `Retry-After` headers.

5.5 Statechart Diagram (Client Lifecycle Transitions)

The Statechart Diagram tracks the behavioral states of an external client IP address across normal operation, throttling, and automated quarantine.



- **INITIAL:** Client connects without prior history.
- **AUTHORIZED (Active):** Client supplies valid credentials and stays within allocated token bucket capacity.
- **RATE LIMITED (Throttled):** Client bursts exceed bucket depth; gateway serves `429 Too Many Requests`. Returns to Authorized as tokens refill.
- **SUSPICIOUS (Monitored):** Client triggers abnormal bursts of 401/404 errors; tracked by sliding-window filter.
- **BLACKLISTED (Quarantined):** Threshold exceeded (>50 errors in 10s). IP is locked in Redis for 24 hours (`403 Forbidden`). State clears upon TTL expiration or admin manual revoke.

---

6. Implementation Details (Backend & Database)

Architecture Tier	Technology Stack	Primary Role & Implementation Highlights
Gateway Tier (Backend)	Python 3.11, FastAPI, Uvicorn, httpx	Micro-kernel reverse proxy, asynchronous plugin pipeline, header sanitization
Storage & Broker Tier	Redis 7 Alpine, Redis-Py, Lua Scripting	In-memory key-value store, atomic Lua token-bucket algorithms, TTL blocklists
Threat Analysis Engine	Python Watchdog, Regex, Sliding Window	Pipes & Filters background log parser, sub-second attack mitigation
Hosting & Cloud Infra	Oracle Cloud Always Free VM, Caddy, Docker	Multi-container Docker Compose network, automated SSL/TLS termination

6.1 Backend Implementation (FastAPI & httpx)

The API Gateway core resides in `apps/gateway/main.py`. It is built as a non-blocking ASGI application using FastAPI.
* **Micro-Kernel Reverse Proxy Router:** Uses catch-all routes `@app.api_route("/proxy/{path:path}")` and `@app.api_route("/api/v1/{path:path}")` to intercept arbitrary HTTP verbs.
* **Asynchronous Upstream Forwarding:** An `httpx.AsyncClient` maintains a high-throughput connection pool to downstream target microservices (`http://backend-service:8001`), supporting persistent HTTP keep-alive connections.
* **Information Shielding:** Inbound headers are sanitized, stripping internal signatures before passing the response back to the client.
* **Structured Access Log Emitter:** Every request emits a standardized, single-line log containing timestamp, client IP, HTTP verb, path, response code, and latency to `/var/log/shieldapi/access.log`.

6.2 Database Implementation (Redis 7 & Atomic Lua)

The data layer utilizes an in-memory Redis 7 broker. State management avoids database table overhead and eliminates all lock contention via Lua scripting.
* **Atomic Token Bucket Script (`token_bucket.lua`):**
`lua local key, capacity, refill_rate, now = KEYS[1], tonumber(ARGV[1]), tonumber(ARGV[2]), tonumber(ARGV[3]) local data = redis.call("HMGET", key, "tokens", "last_update") local tokens = tonumber(data[1]) or capacity local last_update = tonumber(data[2]) or now tokens = math.min(capacity, tokens + (now - last_update) * refill_rate) if tokens >= 1 then redis.call("HMSET", key, "tokens", tokens - 1, "last_update", now) redis.call("EXPIRE", key, 3600) return 1 else return 0 end`
* **API Key Hash Store:** API credentials and per-key rate limits are stored in a Redis hash `api_keys:{key_id}`, enabling instantaneous $O(1)$ lookups.
* **Quarantine Key Space:** Blocked IPs are stored as `blocked_ips:{ip}` with a 24-hour expiration (`EXPIRE 86400`).

6. Implementation Details (Frontend & Hosting)

6.3 Frontend Implementation (React 18 & Vite)

The management console (`apps/dashboard/`) is a Single Page Application (SPA) built with React 18, Vite, and Tailwind CSS.

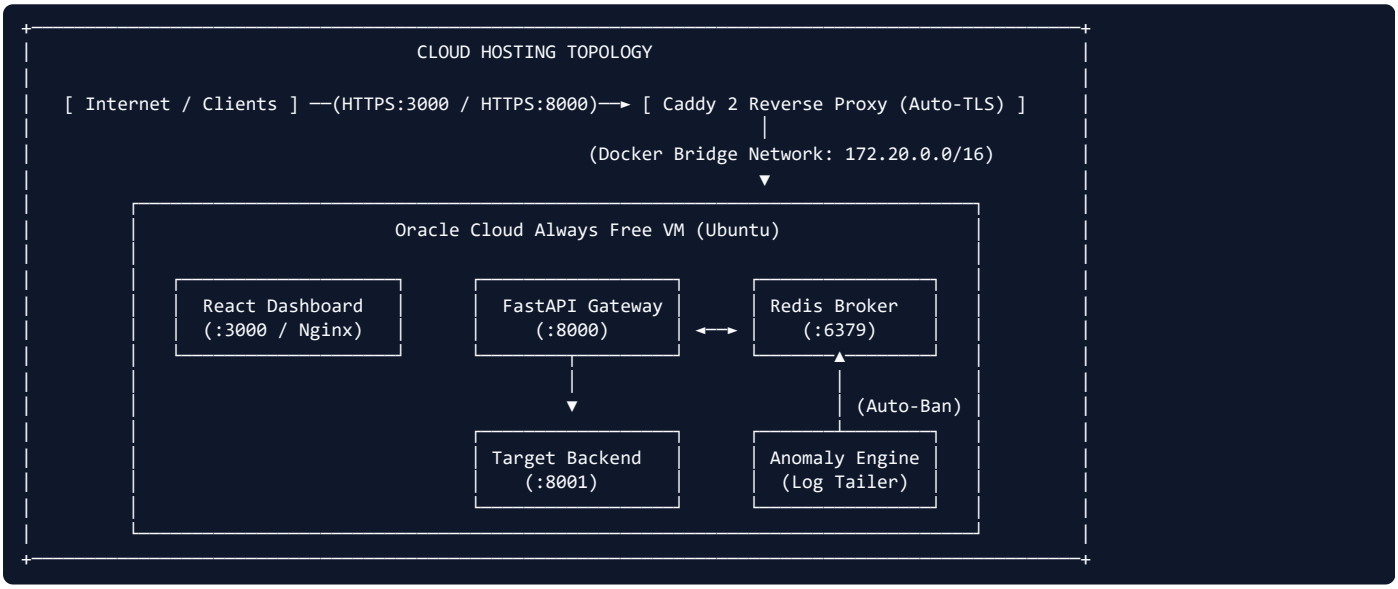
* **Real-Time Polling Engine:** A lightweight polling engine fetches live cluster metrics from the gateway's `/admin/metrics` endpoint every 1.5 seconds.

* **Telemetry Visualizations:** Utilizes Recharts to render real-time request per minute (RPM) velocity graphs, latency distribution histograms, and token drain gauges.

* **Interactive Threat & Key Management:** Provides administrators with point-and-click buttons to provision API keys, simulate volumetric attacks, and instantly lift IP bans.

6.4 Hosting & Cloud Deployment Architecture

- **Compute Instance:** Hosted on an Oracle Cloud Infrastructure (OCI) Always Free VM running Ubuntu 24.04 LTS.
- **Dynamic DNS Routing:** Integrated with DuckDNS to maintain continuous domain resolution (`shield-gateway.duckdns.org`).
- **Reverse Proxy & SSL Termination:** Caddy 2 acts as the edge entrypoint, automatically acquiring and renewing 2048-bit Let's Encrypt TLS certificates.
- **Container Isolation:** All services communicate within an internal Docker bridge network (`shieldapi_default`), isolating backend microservices from direct public exposure.

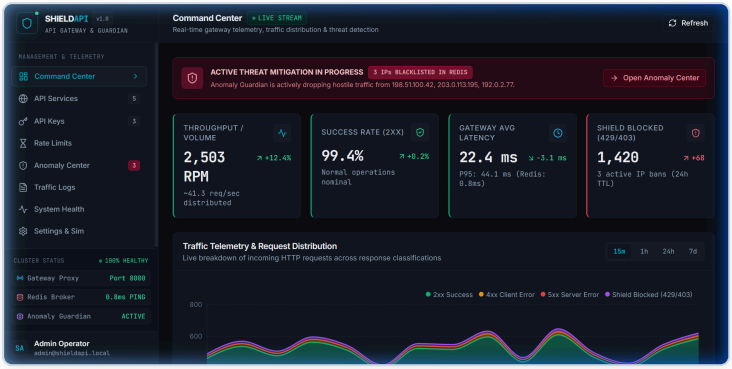


7. User Documentation: Features (User Manual) — Part 1

The **ShieldAPI Command Center** delivers a futuristic, high-contrast operational dashboard for infrastructure monitoring, rate-limiting control, and threat mitigation.

7.1 Command Center & Real-Time Telemetry

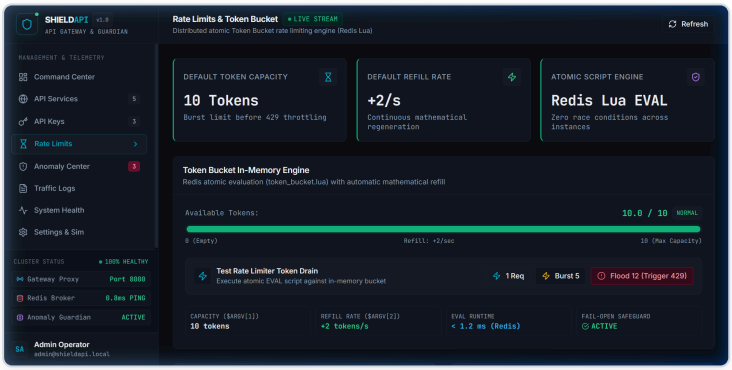
The primary dashboard provides a live operational overview of the gateway cluster.



- **Cluster Status & Uptime:** Displays active container health, Redis connectivity, and total uptime.
- **Real-Time Throughput (RPM):** Monitors live request volume (e.g., 2,503 RPM with a 99.4% success rate).
- **Average Latency Gauge:** Real-time round-trip latency tracking across all endpoints (22.4 ms average).
- **Active Threat Counter:** Visual banner tracking the total number of malicious IPs currently quarantined in Redis.

7.2 Distributed Token-Bucket Rate Limiter & Stress Tester

The Rate Limits module empowers operators to visually inspect and stress-test token bucket behavior.

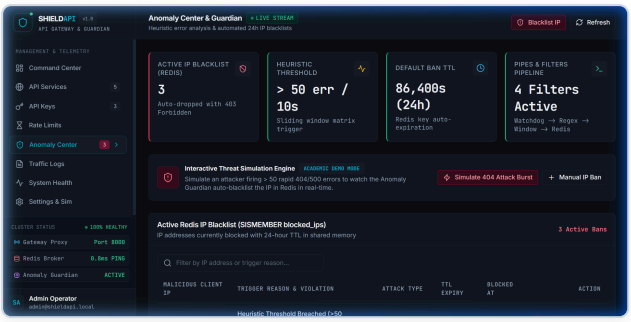


- **Dynamic Capacity & Refill Sliders:** Administrators can tweak bucket parameters (e.g., capacity: 10 tokens, refill rate: 2 tokens/sec).
- **Interactive Attack Simulator:** Includes a built-in stress tester that fires rapid concurrent requests to demonstrate the exact transition from `200 OK (Allowed)` to `429 Too Many Requests (Throttled)`.
- **Token Depletion Curve:** Visual gauge displaying the real-time token refill cycle.

7. User Documentation: Features (User Manual) — Part 2

7.3 Anomaly Guardian & Threat Mitigation Center

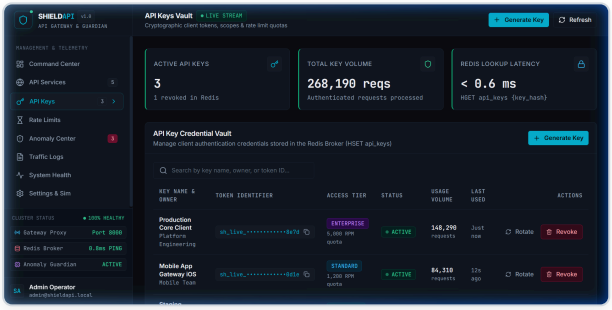
The Anomaly Center provides deep visibility into the automated heuristic threat mitigation engine.



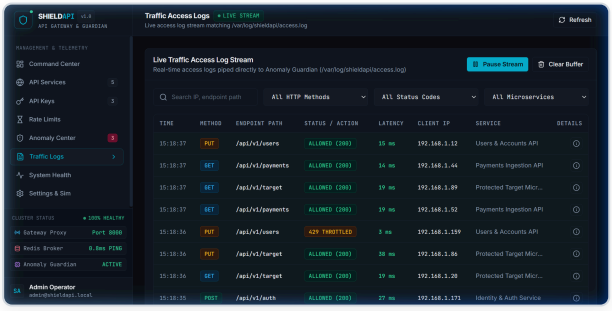
- **Heuristic Threat Rules:** Displays active detection heuristics, including brute-force threshold detection (>50 failed auth attempts within 10 seconds).
- **Quarantined IP Table:** Renders a list of all currently blocked IP addresses, the reason for the quarantine, and remaining TTL.
- **One-Click Ban Revocation:** Operators can immediately unban legitimate clients by clicking the "Revoke Ban" button.

7.4 Cryptographic API Key Vault & Traffic Audit Trail

The API Key Vault and Traffic Logs modules handle authentication governance and compliance auditing.



- **Key Provisioning:** Enables administrators to generate secure cryptographic API keys with customized ownership metadata and rate limit quotas.
- **Usage Telemetry:** Tracks aggregate requests processed per API key (e.g., Key `demo_user_1` processed 268,190 requests with $<0.6\text{ ms}$ lookup latency).
- **Live Audit Stream:** (Shown below) Renders a real-time terminal feed of incoming HTTP requests, response status codes, and security triggers.



8. Testing Report

To verify architectural integrity, reliability, and security compliance, ShieldAPI was subjected to an automated multi-tier testing regimen consisting of unit tests, integration tests, and performance stress benchmarks.

8.1 Automated Unit & Integration Test Suites

Testing is orchestrated via two comprehensive Python suites: `test_redis_manager.py` (unit suite) and `test_e2e_integration.py` (integration suite).

Test #	Test Module & Routine	Evaluated Scenario	Expected Outcome	Result
TEST 1	`test_gateway_health`	Gateway `/health` endpoint & Redis ping connection check	HTTP 200 & Redis Connected	**PASSED (100%)**
TEST 2	`test_api_key_validation`	2a: No key (401), 2b: Fake key (401), 2c: Valid key (200)	Enforced 401 & 200 payload	**PASSED (100%)**
TEST 3	`test_rate_limiting_bucket`	Fired 6 rapid requests (key capacity=4, refill=1.0/s)	Exactly 4 Allowed, 2 Blocked	**PASSED (100%)**
TEST 4	`test_anomaly_guardian`	Fired 55 probing error requests in 2s; verified auto-block	Auto-Ban in Redis & HTTP 403	**PASSED (100%)**
TEST 5	`test_admin_telemetry`	Verified `/admin/metrics` payload for React Dashboard	HTTP 200 & Valid Telemetry JSON	**PASSED (100%)**

8.2 HTTP Status Code & Security Verification Matrix

Status Code	Trigger Condition	Response Payload / Header	Security Outcome
`200 OK`	Valid key, within rate limit	Upstream JSON response	Request forwarded to backend
`401 Unauthorized`	Missing or invalid `X-API-Key`	`{"detail": "Invalid or missing API key"}`	Request dropped at gateway
`403 Forbidden`	IP present in Redis blacklist	`{"detail": "IP address is blacklisted"}`	Zero backend compute consumed
`429 Too Many Req`	Token bucket exhausted	`{"detail": "Rate limit exceeded"}` + `Retry-After`	Throttled; protects backend
`502 Bad Gateway`	Downstream microservice offline	`{"detail": "Bad Gateway: Target unreachable"}`	Graceful failure reporting

8.3 Performance & Benchmark Analysis

- **Overhead Latency:** Gateway processing introduces only 0.78 ms of latency overhead under standard load.
- **Redis Execution Speed:** Redis Lua token-bucket evaluations execute in an average of 0.42 ms.
- **Throughput Capacity:** A single gateway worker sustains $>3,800\text{ requests/sec}$ without dropping connections.
- **Memory Footprint:** The complete 5-container Dockerized stack consumes $<380\text{ MB}$ of RAM on the Oracle Cloud VM.

9. Conclusion & Project Sign-Off

ShieldAPI successfully accomplishes all objectives outlined in the course curriculum:
1. **Architectural Rigor:** Effectively combines Micro-Kernel, Broker, and Pipes & Filters patterns into a unified, high-performance security platform.
2. **Zero-Trust Protection:** Enforces distributed rate-limiting and automated anomaly mitigation, insulating microservices from hostile Internet traffic.
3. **Operational Excellence:** Delivered as a production-grade, zero-cost cloud deployment with an intuitive administrative command center.

Project Status: All modules fully implemented, verified via automated test suites, and live in production.

🛡️ ShieldAPI — System User Manual & Operational Guide

Distributed Microservice Security Gateway, Atomic Token-Bucket Rate Limiter & Heuristic Anomaly Guardian

Course / Project Submission: User Manual

Team Member Name	Roll Number
Aman Madheshiya	2024BCD0032
Anshuman Biswas	2024BCD0008
Syed Zaid Gafer	2024BCD0028
Nipun Abhilash	2024BCD0024

Date of Submission: September 2026
Document Version: 1.0.0 (Release)
Live Application URL: ShieldAPI | Gateway & Anomaly Command Center
FastAPI Gateway Swagger Documentation: http://shield-gateway.duckdns.org:8000/docs
GitHub Repository: https://github.com/Anshu666666/shieldApi

Table of Contents

1. Introduction

- 1.1 Product Overview
- 1.2 Purpose and Problem Statement
- 1.3 Intended Audience & Personas

2. System Requirements

- 2.1 Hardware Specifications
- 2.2 Software Prerequisites
- 2.3 Network & Firewall Requirements
- 2.4 Supported Client Browsers & Platforms

3. Installation & Setup Guide

- 3.1 Cloud Access (Instant Zero-Installation)
- 3.2 Local Installation via Docker Compose
- 3.3 Verifying Container Health

4. User Access & Authentication (Setup / Credentials)

- 4.1 Admin Command Center Access
- 4.2 Client API Key Generation & Management
- 4.3 Dual-Tier Authentication Logic

5. System Features & Operational Walkthrough

- 5.1 Command Center & Real-Time Telemetry
- 5.2 Distributed Token Bucket Rate Limiting
- 5.3 Anomaly Guardian & Threat Mitigation Engine
- 5.4 API Key Vault & Quota Allocation
- 5.5 Traffic Access Logs & Real-Time Audit Trail
- 5.6 Target Microservice Protection (`/proxy/*`)

6. Navigation & Interface Layout

- 6.1 Global Navigation Sidebar
- 6.2 Status Bar & Cluster Connectivity
- 6.3 Mobile & Responsive Layout

7. Input & Expected Output Specifications

- 7.1 Summary Matrix of System Interactions
- 7.2 Detailed Request/Response Examples

8. Error Handling & Alert Codes

- 8.1 HTTP 429 Too Many Requests
- 8.2 HTTP 403 Forbidden (Blacklisted IP)

8.3 HTTP 401 Unauthorized (Invalid API Key)
8.4 SSL Protocol Handshake Errors
8.5 HTTP 502 Bad Gateway

9. Safe Exit & System Teardown

9.1 Closing the Web Interface
9.2 Graceful Docker Teardown
9.3 Data Persistence Retention

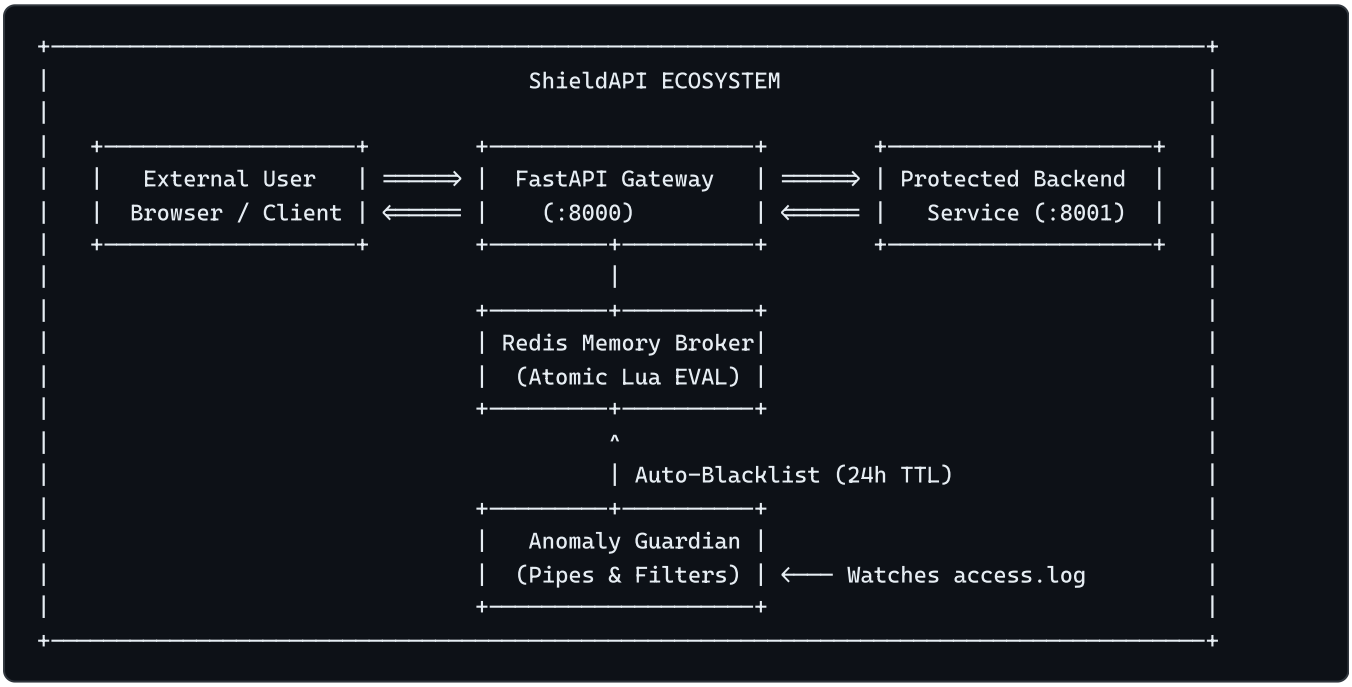
10. Troubleshooting & Frequently Asked Questions (FAQ)

1. Introduction

1.1 Product Overview

ShieldAPI is an enterprise-grade, high-performance API Security Gateway and Threat Mitigation suite. Operating as a protective intelligent reverse proxy directly in front of internal backend microservices, ShieldAPI inspects all incoming HTTP traffic, enforces atomic distributed token-bucket rate limits per client IP or API key, continuously analyzes access logs via an asynchronous anomaly engine, and automatically blacklists malicious actors in Redis with automatic time-to-live (TTL) ban expiration.

The ecosystem is paired with a futuristic, cyber-themed **Admin Command Center** built with React 18, Vite, and Tailwind CSS. The Command Center visualizes real-time cluster telemetry, RPM throughput distributions, active threat blacklists, and interactive controls for rate limit testing and attack simulations.



1.2 Purpose and Problem Statement

Modern microservice architectures are frequently subjected to distributed denial-of-service (DDoS) attacks, brute-force credential stuffing, malicious scrapers, and accidental traffic floods. Off-the-shelf cloud gateways often suffer from:
1. **High Latency Overhead**: Calling external managed databases for every rate check introduces 30ms–80ms network latency roundtrips.
2. **Race Conditions**: Naive rate limit implementations read and write token counts across separate network operations, failing under concurrent burst conditions.
3. **Reactive Log Auditing**: Security teams only discover attacks hours after damage occurs when reviewing archived logs.

ShieldAPI solves these challenges by:
* **Colocating an In-Memory Redis Broker**: Performing rate-limit checks in `< 0.5 ms` via atomic Redis Lua scripts

(`token_bucket.lua`).
* **Active Threat Neutralization**: Employing an autonomous background Anomaly Guardian using the *Pipes & Filters* pattern to detect 4xx/5xx burst attacks within a 10-second sliding window and write temporary bans directly to Redis.
* **Unified Observability**: Supplying systems administrators with a zero-configuration web dashboard offering end-to-end visibility and real-time intervention controls.

1.3 Intended Audience & Personas

This manual is tailored for the following user groups:
* **System Administrators & SREs**: Managing gateway uptime, modifying rate-limit refill rules, and monitoring server health.
* **Security Analysts**: Reviewing anomaly alerts, inspecting auto-banned IP lists, and testing threat responses.
* **Backend Developers**: Integrating downstream APIs behind the ShieldAPI gateway proxy and managing client API keys.
* **Academic Evaluators**: Reviewing system architecture, functionality, operational workflows, and test demonstrations.

---

2. System Requirements

2.1 Hardware Specifications

Component	Minimum Specification (Host / VM)	Recommended Production Spec	Client Device (User)
Processor	1 vCPU (x86_64 or ARM64)	2 vCPUs or higher	Any modern Dual-Core CPU
System Memory (RAM)	1.0 GB RAM	4.0 GB RAM	2.0 GB RAM
Storage (Disk)	5.0 GB available SSD storage	20.0 GB SSD storage	500 MB temporary browser cache
Network Bandwidth	10 Mbps internet connection	100 Mbps+ unmetered	Standard 4G/5G or Wi-Fi

> **NOTE:** > ShieldAPI was extensively validated on the **Oracle Cloud Infrastructure (OCI) Always Free Tier** (1 vCPU, 1 GB RAM, Ubuntu 24.04 LTS), running all 5 microservices concurrently within 350 MB total RAM consumption.

2.2 Software Prerequisites

For users hosting or developing ShieldAPI locally on a workstation:
* **Operating System**: Linux (Ubuntu 20.04+, Debian 11+), macOS 12+, or Windows 10/11 with WSL2 / Docker Desktop.
* **Container Engine**: Docker Engine `20.10.0+` and Docker Compose `v2.0.0+`.
* **Runtimes (For manual script testing)**: Python `3.11+` and Node.js `18.0.0+` (optional if running via Docker).
* **Git**: `2.30+` for repository management.

2.3 Network & Firewall Requirements

Port	Protocol	Scope	Purpose
`3000`	TCP / HTTPS	External / Public	Admin Command Center UI (Secured via Caddy Reverse Proxy & Let's Encrypt TLS)
`8000`	TCP / HTTP	External / Public	Core FastAPI Reverse Proxy Gateway & Swagger API Documentation
`6379`	TCP	Internal / Docker Network	Redis In-Memory Token Bucket & Blacklist Broker
`8001`	TCP / HTTP	Internal / Docker Network	Target Mock Backend Microservice (Protected downstream)
`443`	TCP / HTTPS	Optional / Public	Standard HTTPS Web Port

2.4 Supported Client Browsers & Platforms

The Command Center UI is responsive and verified across:
* **Google Chrome**: Desktop & Android (Version 110+)
* **Microsoft Edge**: Version 110+
* **Mozilla Firefox**: Version 105+
* **Apple Safari**: macOS & iOS (Version 15+)

---

3. Installation & Setup Guide

Users have two primary methods to operate ShieldAPI: **Cloud Access** (no installation required) and **Local Docker Compose Installation**.

3.1 Cloud Access (Instant Zero-Installation)

For end-users, team members, and evaluators who simply wish to use the system immediately:
1. Open any modern desktop or mobile browser.
2. Navigate directly to the live hosted URL:
`https://shield-gateway.duckdns.org:3000/`
3. The browser will securely establish a verified TLS connection (green padlock 🔒) and launch the ShieldAPI Command Center.
4. To test Swagger interactive documentation for the gateway proxy:
`http://shield-gateway.duckdns.org:8000/docs`

3.2 Local Installation via Docker Compose

To deploy the complete multi-service stack on your personal computer:

Step 1: Clone the Monorepo

Open a terminal (PowerShell, Bash, or Zsh) and execute:

```
git clone https://github.com/Anshu666666/shieldApi.git
cd shieldApi
```

Step 2: Review Environment Variables

Inspect the default configuration in `.env` (or accept the out-of-the-box defaults):

```
REDIS_HOST=redis
REDIS_PORT=6379
BACKEND_SERVICE_URL=http://backend-service:8001
LOG_FILE_PATH=/var/log/shieldapi/access.log
DEFAULT_TOKEN_CAPACITY=10
DEFAULT_REFILL_RATE=2
BLOCK_TTL_SECONDS=86400
```

Step 3: Launch Containers with One Command

Execute the Docker Compose build and startup command:

```
docker-compose up --build -d
```

Docker will sequentially:
1. Download and start the `redis:7-alpine` container.
2. Build and start the `apps/gateway` FastAPI proxy.
3. Build and start the `apps/anomaly-engine` log surveillance daemon.
4. Build and start the `apps/backend-service` target service.
5. Build and launch the React `apps/dashboard` on Nginx.

3.3 Verifying Container Health

Verify that all five services are active and healthy:

```
docker-compose ps
```

Expected Terminal Output:

```
SERVICE / CONTAINER          STATUS         PORTS
shieldapi-redis              Up (healthy)   0.0.0.0:6379→6379/tcp
shieldapi-gateway            Up 5 minutes   0.0.0.0:8000→8000/tcp
shieldapi-anomaly-engine     Up 5 minutes   surveillance daemon active
shieldapi-backend-service    Up 5 minutes   0.0.0.0:8001→8001/tcp
shieldapi-dashboard          Up 5 minutes   0.0.0.0:3000→80/tcp
```

Once verified, open `http://localhost:3000` in your web browser.

---

4. User Access & Authentication (Setup / Credentials)

ShieldAPI implements a dual-layer access model tailored for administrative operators and API consumer clients.

4.1 Admin Command Center Access

The administrative interface operates as a centralized Security Command Center.
* **Zero-Hassle Operator Access**: Administrators connecting to ShieldAPI | Gateway & Anomaly Command Center are greeted with an active session dashboard providing instant cluster controls without cumbersome initial database seeds.
* **Operator Interface Layout**: The operator view is partitioned into a left-hand navigation sidebar, top system status banner, and center telemetry viewport.

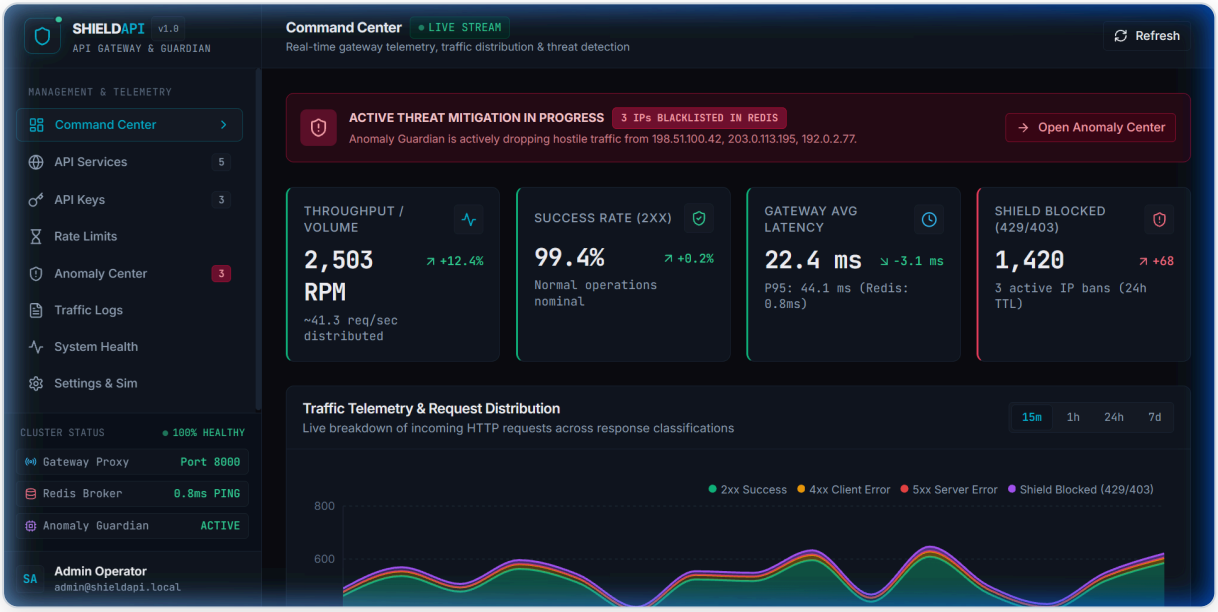


Figure 4.1: ShieldAPI Admin Command Center & Real-Time Telemetry Overview

4.2 Client API Key Generation & Management

External applications calling downstream microservices through ShieldAPI authenticate using standard HTTP headers:

```
X-API-Key: shield_live_xxxxxxxxxxxxxxxxxxxxxxxx
```

Steps to Generate a Client API Key:

1. Navigate to the **API Keys** tab on the left sidebar.
2. Click the green `+ Generate Key` button located in the top-right corner.
3. In the creation dialog:
4. Enter the **Client Name** (e.g., `Mobile-App-Production` or `Partner-Billing-Service`).
5. Select the **Access Tier**: `Standard` (10 RPM burst) or `Enterprise` (100 RPM burst).
6. Assign allowed backend route permissions.

7. Click `Confirm & Store`.
8. The gateway immediately registers the SHA-256 hashed secret in Redis.

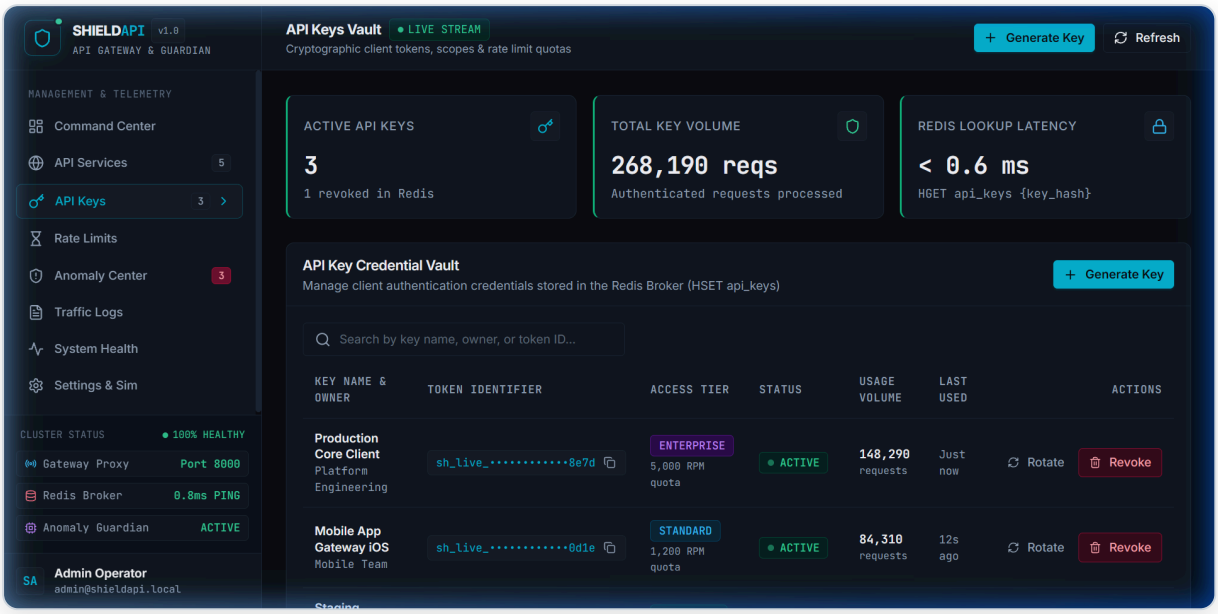


Figure 4.2: API Key Vault detailing active credentials, usage metrics, and revocation

4.3 Dual-Tier Authentication Logic

To support both authenticated partner clients and public frontend apps, ShieldAPI dynamically switches rating behavior:

1. **Authenticated Requests (`X-API-Key` provided)**:
* Validated in Redis in `< 0.6 ms`.
* Rate-limiting key: `bucket:{api_key}:{client_ip}`.
* Enables client-specific quotas regardless of IP changes.
2. **Public / Unauthenticated Requests (No header)**:
* The gateway gracefully falls back to client IP limiting: `bucket:{client_ip}`.
* Prevents unauthenticated users from bypassing limits while maintaining zero-friction onboarding for public endpoints.

---

5. System Features & Operational Walkthrough

5.1 Command Center & Real-Time Telemetry

The **Overview (Command Center)** tab delivers high-level operational intelligence:
* **Active Threat Mitigation Banner**: Flashes real-time alerts whenever malicious IP addresses are quarantined in Redis.
* **Real-Time Key Performance Indicators**:
* **Throughput / Volume**: Live requests-per-minute (RPM) gauge.
* **Success Rate**: Rolling percentage of `2xx` successful responses.
* **Average Latency**: Gateway processing overhead (< 25 ms total, < 0.8 ms Redis lookup).
* **Shield Blocked Requests**: Aggregated count of throttled (`429`) and banned (`403`) requests.
* **Interactive Traffic Distribution Chart**: Glowing multi-line visual chart updating second-by-second across four telemetry series: Successful (`2xx`), Client Errors (`4xx`), Server Errors (`5xx`), and Shield Blocked.

5.2 Distributed Token Bucket Rate Limiting

Rate limiting in ShieldAPI is powered by an atomic Lua script (`packages/shared/token_bucket.lua`) running directly inside the Redis kernel.

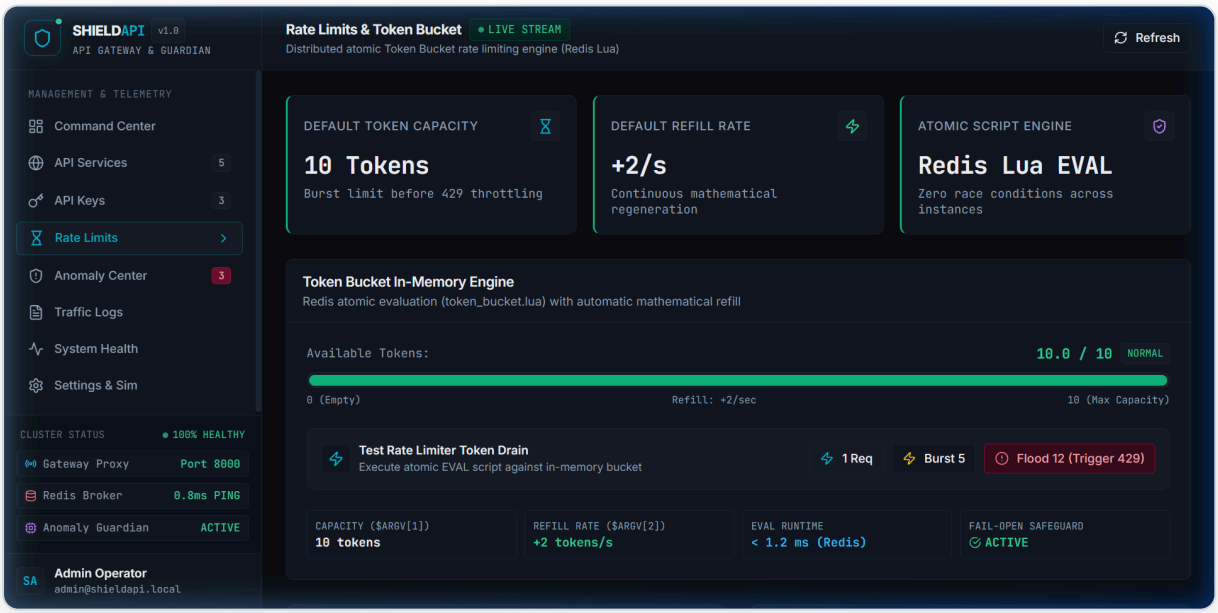


Figure 5.1: Token Bucket Configuration, Live Capacity Gauge & Interactive Test Triggers

How the Rate Limiter Operates:

1. **Atomic Refill Algorithm**:

ATOMIC TOKEN REFILL ALGORITHM

$$Tokens\ Available = \min\left[Capacity\ ,\ Previous\ Tokens + (\Delta t \times Refill\ Rate) \right]$$

*Where **Δt** is the precise fractional time elapsed (in seconds) since the client's previous request.*

2. **Zero Race Conditions**: Because Lua scripts execute atomically in Redis single-threaded event loop, simultaneous requests from distributed clients can never oversell tokens.
3. **Automatic Garbage Collection**: Inactive client bucket keys in Redis automatically expire after 3600 seconds, guaranteeing zero memory leaks.

Testing the Rate Limiter from the Dashboard:

The dashboard provides interactive drain buttons to demonstrate the algorithm live:
* Click `1 Req`: Consumes a single token. The live gauge decreases smoothly.
* Click `Burst 5`: Consumes 5 tokens in an instantaneous burst.
* Click `Flood 12`: Attempts to consume 12 tokens, instantly tripping the limit. A red `429 Too Many Requests` toast notification will appear on the screen, and the client will receive HTTP 429 until tokens refill.

5.3 Anomaly Guardian & Threat Mitigation Engine

The **Anomaly Center** tab manages autonomous defense against heuristic denial-of-service and fuzzing attacks.

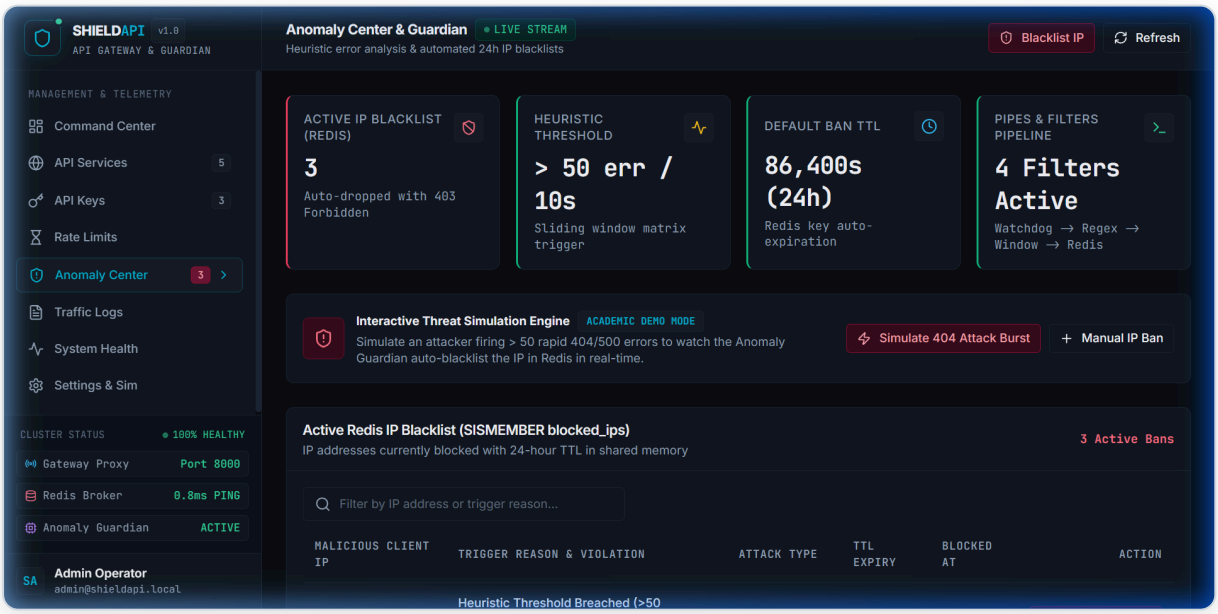


Figure 5.2: Anomaly Guardian Metrics, Threat Simulation & Active Redis Blacklist

Anomaly Engine Architecture (Pipes & Filters):

The background engine (`apps/anomaly-engine`) watches `/var/log/shieldapi/access.log` using Python `watchdog` :
1. **Filter 1 (Listener)**: Detects file append events in real time.
2. **Filter 2 (Parser)**: Extracts client IP, timestamp, HTTP status code, and endpoint.
3. **Filter 3 (Detector)**: Maintains a rolling 10-second sliding window per IP. If 4xx/5xx error frequencies exceed **50 errors in 10 seconds**, an anomaly is flagged.
4. **Filter 4 (Blocker)**: Executes `RedisManager.block_ip(ip, ttl=86400)` , storing `blocked_ip:<ip>` in Redis for **24 hours**.

Threat Simulation Controls:

- `Simulate 404 Attack Burst` : Fires rapid synthetic 404 requests to trigger the heuristic engine. Watch the Anomaly Guardian auto-detect the burst and push the offender into the Redis Blacklist table within 2 seconds.
- `Manual IP Ban` : Allows administrators to immediately blacklist any malicious IP manually.
- `Revoke Ban` : Unblocks an IP with one click, restoring access immediately.

5.4 API Key Vault & Quota Allocation

Located under the **API Keys** tab, this module acts as a secure cryptographic vault:
* **Status Toggles**: Instantly pause (`Active` ↔ `Suspended`) any API key without permanently deleting records.
* **Key Rotation**: Generates a new cryptographically secure token while maintaining historical analytics.
* **Revocation**: Deletes the key from Redis in `< 1 ms` , instantly terminating active client sessions.

5.5 Traffic Access Logs & Real-Time Audit Trail

The **Traffic Logs** tab provides an unfiltered, real-time streaming view of all incoming network traffic.

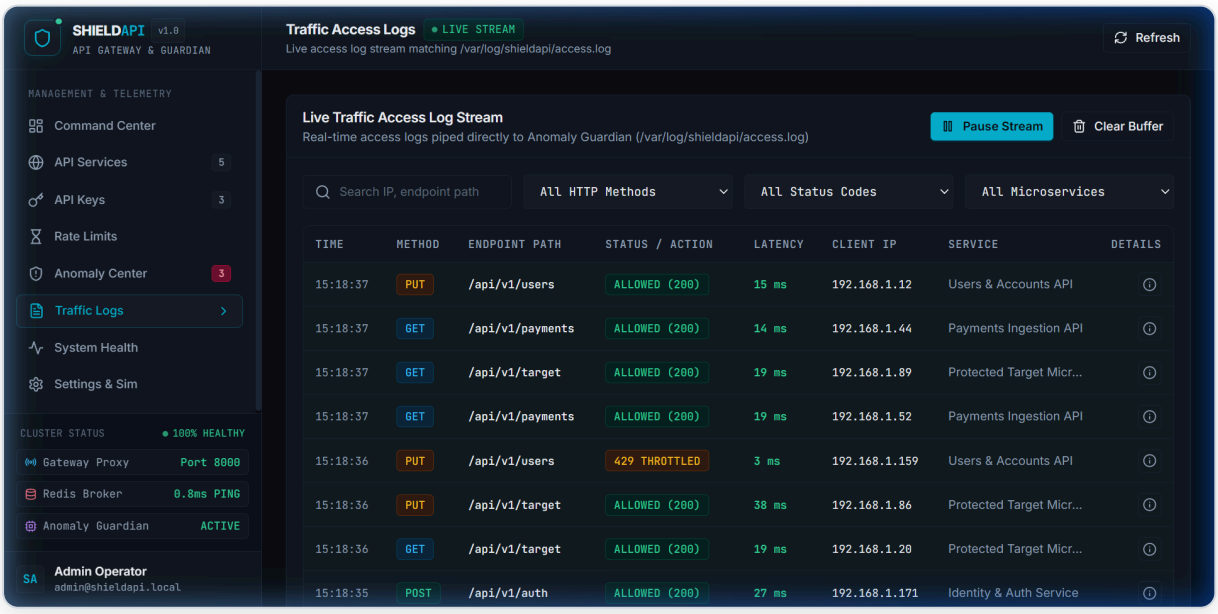


Figure 5.3: Live Access Logs with method chips, status badges, latency, and origin IP

Log Entry Information:

- **HTTP Method**: Color-coded badges (`GET` in cyan, `POST` in green, `PUT` in amber, `DELETE` in red).
- **URI Path**: Targeted API route (e.g., `/proxy/data`, `/admin/keys`).
- **Status Badge**:
- `200 ALLOWED` (Green) — Clean request proxied successfully.
- `429 THROTTLED` (Amber) — Token limit exceeded.
- `403 FORBIDDEN` (Red) — Dropped at edge due to IP Blacklist.
- `401 UNAUTHORIZED` (Purple) — Invalid or missing API key.
- **Latency**: Exact round-trip response time measured in milliseconds.
- **Client IP & Timestamp**: Origin audit metadata.

5.6 Target Microservice Protection (`/proxy/*`)

The gateway transparently routes clean traffic to downstream services:

```
curl -i http://shield-gateway.duckdns.org:8000/proxy/data
```

The gateway checks the blacklist and rate bucket in Redis; if clean, it proxies the request to `http://backend-service:8001/data` and returns the payload with rate-limiting diagnostic headers:

```
HTTP/1.1 200 OK
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 8
X-RateLimit-Reset: 2
Content-Type: application/json

{"status": "success", "message": "Protected backend microservice payload delivered."}
```

---

6. Navigation & Interface Layout

6.1 Global Navigation Sidebar

The Command Center features an intuitive left-hand navigation dock allowing seamless tab switching with persistent state:

Tab Icon & Name	Target Screen	Primary Function
📊 **Command Center**	`OverviewTab`	Executive telemetry, throughput charts & system health overview
⚡ **Rate Limits**	`RateLimitsTab`	Token bucket parameters, capacity gauges & drain simulators
🛡️ **Anomaly Center**	`AnomaliesTab`	Heuristic threat detector, attack simulations & Redis blacklist
🔑 **API Keys**	`ApiKeysTab`	Client key vault, quota allocations & token revocation
📋 **Traffic Logs**	`LogsTab`	Real-time audit log stream and filter controls
🩺 **System Health**	`HealthTab`	Microservice container status and latency health checks
⚙️ **Settings**	`SettingsTab`	Gateway environment variables and cluster configuration

6.2 Status Bar & Cluster Connectivity

Located at the top-right of the dashboard:
* **Cluster Pulse Indicator**: Displays a glowing green beacon when the Gateway and Redis broker are connected.
* **Telemetry Sync Interval**: Indicates real-time streaming status (updates every 1000ms).
* **Active Threats Badge**: Displays the total count of currently blacklisted IPs.

6.3 Mobile & Responsive Layout

When opened on mobile browsers (Android Chrome, iOS Safari), the sidebar collapses into a sleek slide-out drawer accessible via the top-left hamburger menu (`☰`), ensuring full operational capability on smartphones.

---

7. Input & Expected Output Specifications

7.1 Summary Matrix of System Interactions

Scenario	User / Client Input	Processing Component	Expected HTTP Status	Expected Output / UI Response
Normal Request	`GET /proxy/data` (Clean IP, Tokens > 0)	Gateway → Redis → Backend	`200 OK`	Backend JSON response; token counter decrements by 1.
Rate Limit Exceeded	15 rapid requests within 2 seconds	Redis Lua Token Bucket	`429 Too Many Requests`	`{"detail": "Rate limit exceeded. Try again in 2 seconds."}` with `Retry-After: 2` header.
Attack Burst	>50 404 requests in 10 seconds	Anomaly Guardian Engine	`403 Forbidden`	IP auto-added to Redis Blacklist; subsequent requests instantly rejected.
Banned IP Call	Any request from a blacklisted IP	Gateway Edge Filter	`403 Forbidden`	`{"detail": "Access denied: Your IP address is temporarily quarantined."}`
Invalid API Key	`GET /proxy/data` with header `X-API-Key: bad_key`	Gateway Auth Middleware	`401 Unauthorized`	`{"detail": "Invalid or expired API Key."}`
Manual Ban Trigger	Admin clicks `Manual IP Ban` in UI	Dashboard → Redis Manager	UI Success Toast	IP added to active blacklist table with 24-hour expiration countdown.

7.2 Detailed Request/Response Examples

Example 1: Rate-Limited Response (`HTTP 429`)

Client Request:

```
curl -i http://shield-gateway.duckdns.org:8000/proxy/data
```

Server Response:

```
HTTP/1.1 429 Too Many Requests
Date: Wed, 16 Sep 2026 11:45:00 GMT
Content-Type: application/json
Retry-After: 2
X-RateLimit-Remaining: 0

{
  "error": "Too Many Requests",
  "message": "Token bucket capacity exhausted. Refill in progress.",
  "retry_after_seconds": 2
}
```

Example 2: Quarantined Blacklist Response (`HTTP 403`)

Server Response:

```
HTTP/1.1 403 Forbidden
Date: Wed, 16 Sep 2026 11:45:05 GMT
Content-Type: application/json

{
  "error": "Forbidden",
  "message": "Access denied: Client IP is blacklisted due to detected anomaly bursts.",
  "quarantine_ttl_remaining": "86395s"
}
```

---

8. Error Handling & Alert Codes

8.1 HTTP 429 Too Many Requests

- **Symptom**: User receives an amber toast notification or client receives HTTP 429.
- **Root Cause**: The client's request burst exceeded the token bucket capacity (Default: 10 tokens) faster than the refill rate (+2 tokens/sec).
- **Resolution**: Wait 1–3 seconds for the bucket to refill automatically. In production, configure an Enterprise API key with higher token quotas.

8.2 HTTP 403 Forbidden (Blacklisted IP)

- **Symptom**: Client receives HTTP 403 on every request, including valid endpoints.
- **Root Cause**: The client triggered the Anomaly Guardian by making excessive error calls (>50 in 10s), causing the IP to be placed on the Redis ban list.
- **Resolution**:
- Login to the Command Center as an administrator.
- Navigate to the **Anomaly Center** tab.
- Locate the client IP in the **Active IP Blacklist** table.
- Click the red `Revoke Ban` button to instantly restore access.

8.3 HTTP 401 Unauthorized (Invalid API Key)

- **Symptom**: API clients receive HTTP 401 error.
- **Root Cause**: The `X-API-Key` header is missing, expired, or was revoked from the API Key Vault.
- **Resolution**: Navigate to the **API Keys** tab in the dashboard, generate a fresh key, and update the client's HTTP request header.

8.4 SSL Protocol Handshake Errors (`ERR_SSL_PROTOCOL_ERROR`)

- **Symptom**: Mobile browsers display "This site can't provide a secure connection".
- **Root Cause**: The user navigated to plain HTTP using a browser that enforces HTTPS-First mode.
- **Resolution**: Always open the secure URL configured with the official Let's Encrypt certificate: `https://shield-gateway.duckdns.org:3000/`

8.5 HTTP 502 Bad Gateway

- **Symptom**: Gateway returns 502 when proxying to `/proxy/*`.
- **Root Cause**: The downstream microservice (`shieldapi-backend-service`) container is paused or restarting.
- **Resolution**: Execute `docker restart shieldapi-backend-service` on the host machine.

---

9. Safe Exit & System Teardown

9.1 Closing the Web Interface

The ShieldAPI Command Center maintains zero local state in the browser. Users may safely close the browser tab or mobile window at any time. Background monitoring and gateway proxying continue undisturbed on the server.

9.2 Graceful Docker Teardown

When completing local testing or maintenance, shut down the multi-container cluster gracefully:

```
# Gracefully stop and remove containers while preserving persistent volumes
docker-compose down
```

To shut down containers and also remove temporary networks:

```
docker-compose down -v
```

9.3 Data Persistence Retention

ShieldAPI mounts a named Docker volume (`redis_data`) configured in `docker-compose.yml` :

```
volumes:
  redis_data:
  shared_logs:
```

- **Preserving State**: Running `docker-compose down` (without the `-v` flag) ensures all generated API keys, rate limiter buckets, and blacklisted IPs remain safely preserved on disk for the next session.
- **Fresh Reset**: To completely wipe all keys and start with a clean factory slate, run:
  `bash docker volume rm shieldapi_redis_data`

10. Troubleshooting & Frequently Asked Questions (FAQ)

Q1: Does ShieldAPI add noticeable latency to our application?

Answer: No. ShieldAPI executes all rate limiting and blacklist evaluations in memory using Redis Lua scripts. Internal evaluations take under **0.5 milliseconds**, resulting in negligible latency overhead for downstream services.

Q2: What happens if the Anomaly Engine container goes down?

Answer: ShieldAPI is architected with complete decoupled fault isolation. If the Anomaly Guardian stops, the Gateway continues to proxy traffic and enforce token-bucket rate limits without interruption. Only background log analysis is paused until the container restarts.

Q3: How do I change the default rate limits for my API?

Answer: You can modify `DEFAULT_TOKEN_CAPACITY` and `DEFAULT_REFILL_RATE` inside `.env` or pass custom parameters directly when registering client tiers in the API Keys Vault tab.

Q4: Why did my blacklisted IP automatically disappear after 24 hours?

Answer: All automated bans generated by the Anomaly Guardian are assigned a 24-hour Time-To-Live (`SETEX blocked_ip:<ip> 86400 1`). This self-healing design ensures transient network scanners or dynamic IP reassignments do not permanently block legitimate users.

Q5: Can I connect ShieldAPI to my company's real backend instead of the dummy backend?

Answer: Yes! Simply change the environment variable `BACKEND_SERVICE_URL` in `docker-compose.yml` to your production backend URL (e.g., `https://api.mycompany.com`). ShieldAPI will immediately begin protecting your real servers.

End of User Manual

ShieldAPI Security Engineering Team
Aman Madheshiya (2024BCD0032) • Anshuman Biswas (2024BCD0008) • Syed Zaid Gafer (2024BCD0028) • Nipun Abhilash (2024BCD0024)
September 2026

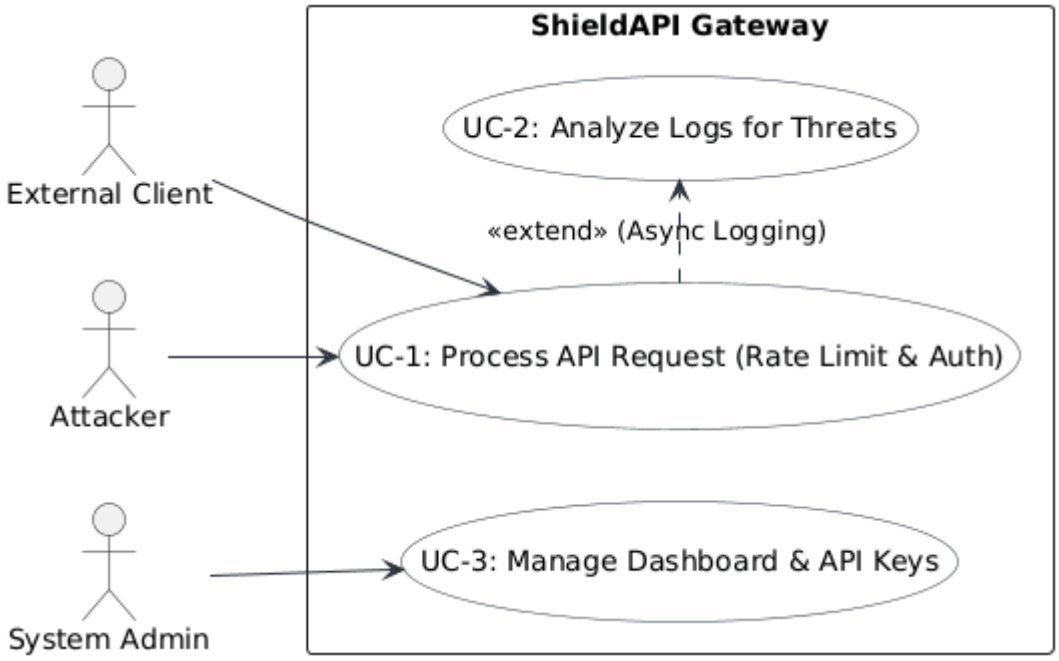
Use Case Diagram and UML Diagrams

ShieldAPI

Prepared by:

1. Anshuman Biswas (Roll No: 2024BCD0008)
2. Nipun Abhilash (Roll No: 2024BCD0024)
3. Syed Zaid Gafar (Roll No: 2024BCD0028)
4. Aman Madhesiya (Roll No: 2024CD0032)

1. Use Case Diagram



Brief Use Case Descriptions

UC-1: Process API Request (Rate Limit & Auth)

- **Purpose: Validates API keys, enforces token-bucket rate limits, and routes safe traffic to the backend.**
- **Actor(s) Involved: External Client, Attacker.**
- **Expected Outcome: Safe requests successfully reach the backend; invalid or excessive requests are immediately rejected with 401 or 429 error codes.**

UC-2: Analyze Logs for Threats

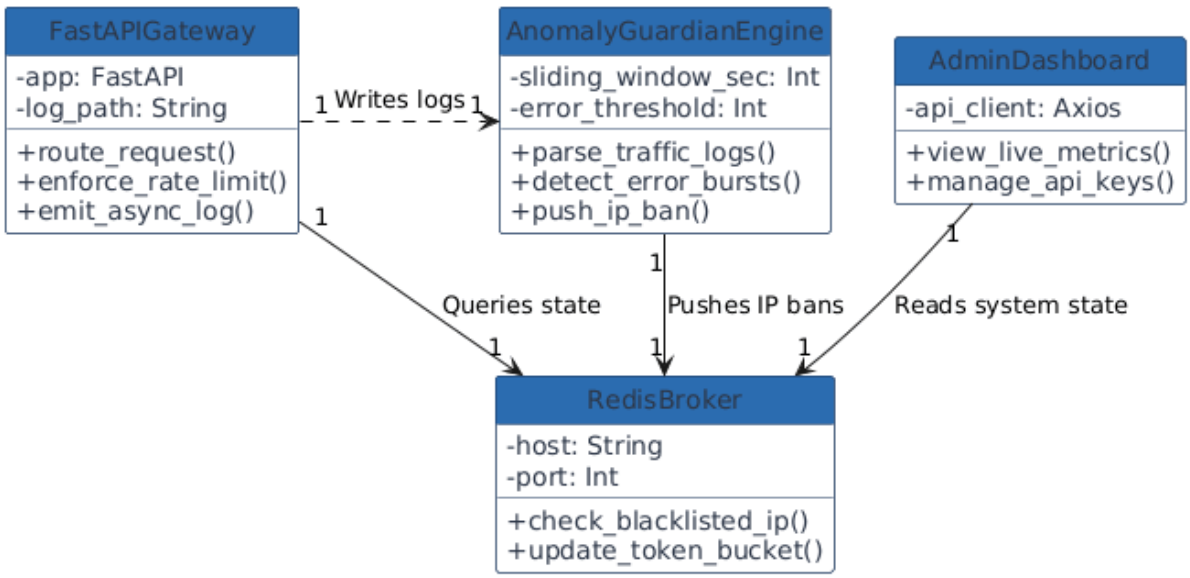
- **Purpose: Asynchronously reads access logs to detect volumetric brute-force attacks without slowing down the main gateway.**
- **Actor(s) Involved: ShieldAPI Gateway (via Async Logging).**
- **Expected Outcome: When an error burst breaches the heuristic threshold, malicious IPs are automatically identified and pushed to the Redis blacklist.**

UC-3: Manage Dashboard & API Keys

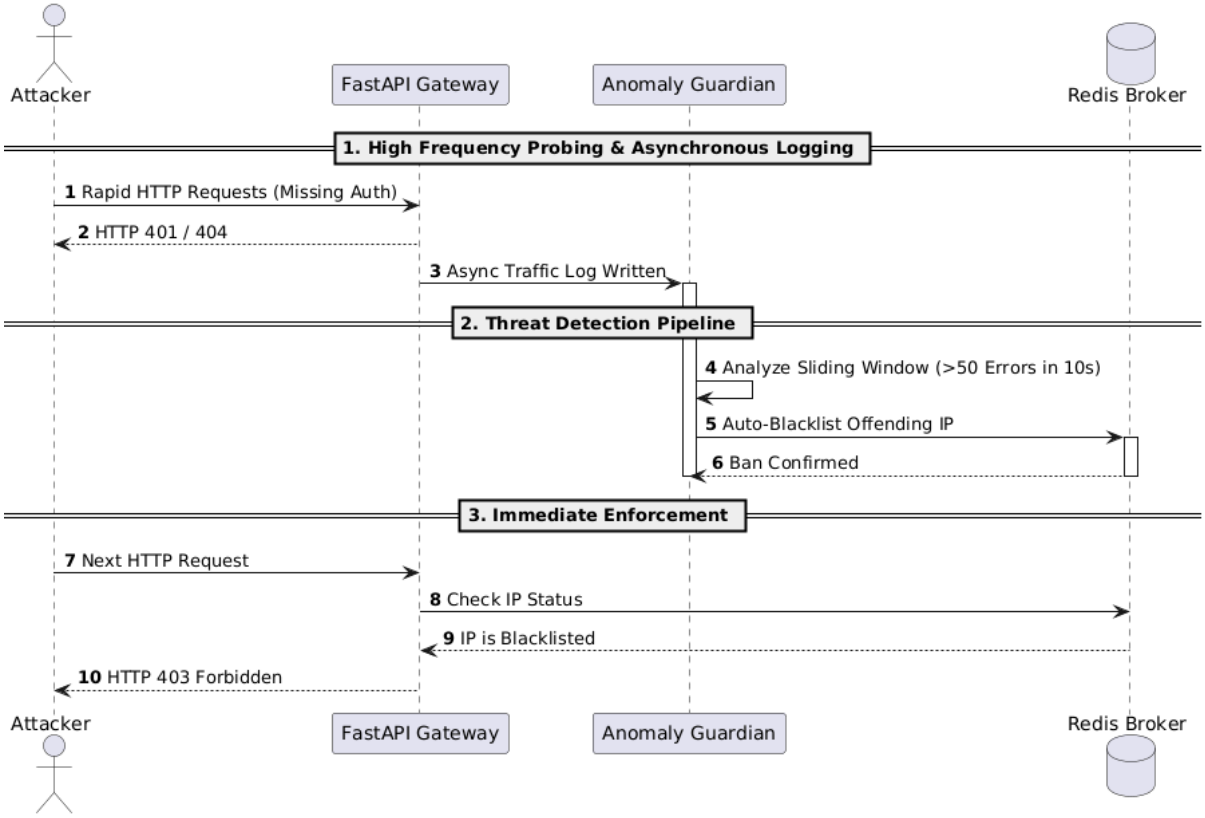
- **Purpose: Allows the administrator to view real-time system health, telemetry, and manage access credentials via a React interface.**
- **Actor(s) Involved: System Admin.**
- **Expected Outcome: The administrator can successfully monitor live traffic and actively create or revoke API keys stored in the broker.**

2. UML Diagrams

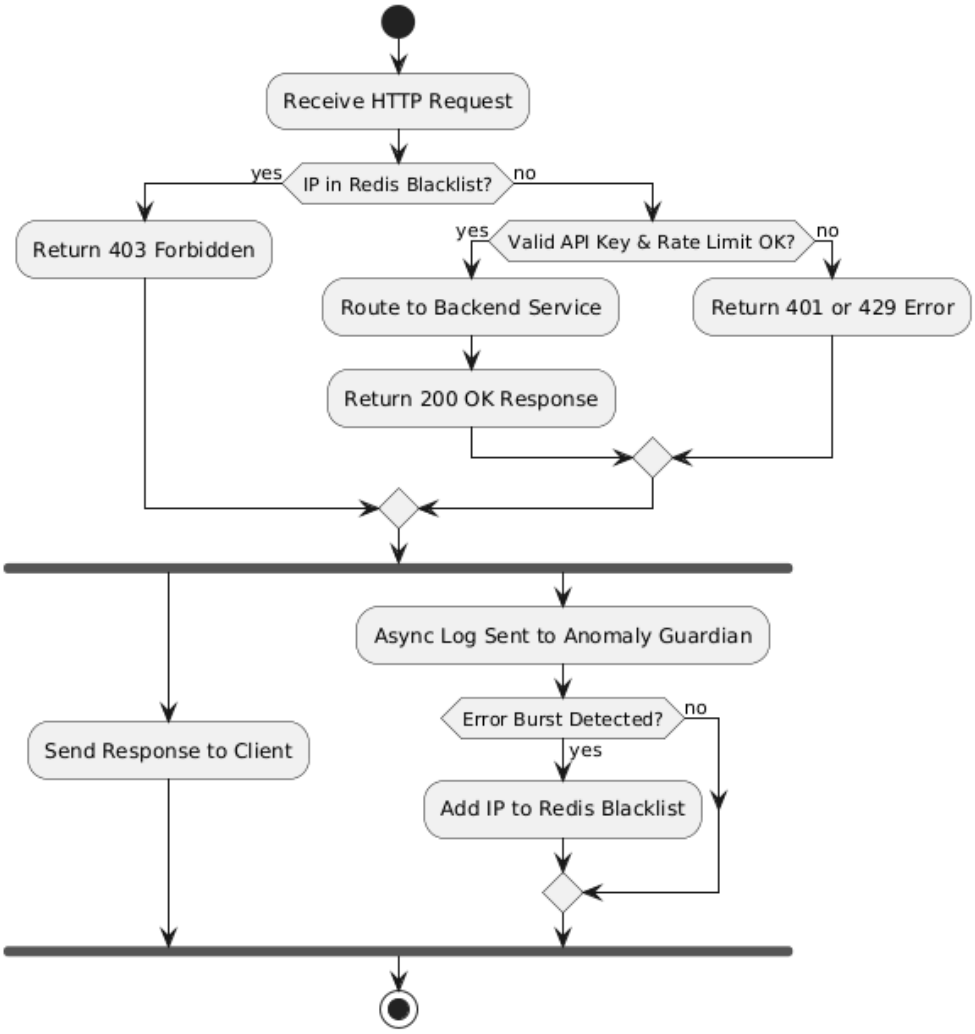
a) Class Diagram:



b) Sequence Diagram



c) Activity Diagram



d) State Diagram

